UNIVERSITÀ DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale in Ingegneria Informatica

Integrazione del Protocollo TEC Tree per la Sicurezza e Autenticazione della Memoria Off-Chip con il Protocollo di Coerenza Tardis-TSO

Anno Accademico 2025/2026

Relatore
Ch.mo prof. Alessandro Cilardo

Candidato
Fuorto Simone
matr. M63001564

Abstract

Con l'aumentare della gamma di servizi forniti dai sistemi embedded, cresce anche la quantità di dati sensibili che essi manipolano. Di conseguenza, esistono attualmente forti incentivi per gli aggressori che desiderano trarre profitto da attacchi a questi sistemi. Gli attacchi sui bus consentono di recuperare i dati in transito da o verso la memoria, sollevando il problema della riservatezza dei dati. In generale, per garantire la riservatezza vengono implementati schemi di cifratura. Tuttavia, nella maggior parte dei casi, la sola crittografia non è sufficiente quando un avversario è in grado di manipolare i dati senza essere rilevato (vedi *bus probing*). L'integrità della memoria è quindi un obbiettivo primario per la protezione di una piattaforma informatica contro attaccanti attivi. L'obbiettivo di questo lavoro è l'implementazione di un protocollo in grado di garantire integrità e confidenzialità grazie alla cifratura della memoria *off-chip* basata su *Tamper-Evident Counter Tree* e di valutarne le performance in relazione ai costi di realizzazione introdotti.

Indice

Abstract **i**

1 Fondamenti Teorici **1**

1.1 Contesto e Motivazioni 1

1.2 Autenticazione della Memoria 3

1.2.1 Autenticazione basata su Contatori 3

1.2.2 Tamper Evident Counter Tree 5

1.3 Il Protocollo di Coerenza Tardis-TSO 6

1.3.1 Il modello TSO (Total Store Order) 7

1.3.2 Il meccanismo dei Timestamp logici 7

1.3.3 Architettura del Protocollo 8

1.3.4 Integrazione dell'Autenticazione e Coerenza . . 9

1.4 Ambiente di Simulazione GEM5 11

1.4.1 Modulo Ruby 11

1.5 Obbiettivi e Workflow della Tesi 12

2 Implementazione del Protocollo Sicuro **14**

2.1 Modellazione Fisica dell'Integrazione 15

2.1.1 Partizionamento dello Spazio Fisico 19

2.1.2 Evoluzione Strutturale della Directory 20

2.2 Modifica dell'Automa in SLICC 23
2.2.1 Nuovi Stati Architetturali 23
2.2.2 Estensione degli Eventi 26
2.2.3 Automa a Stati Finiti: Data Chunk 29
2.2.4 Automa a Stati Finiti: Counter chunk 32
2.2.5 Riepilogo delle Transizioni dell'Automa SLICC 34
2.3 Interfaccia di Rete e Code SLICC 37
2.3.1 Inbound Queue 38
2.3.2 Outbound Queue 43

3 Valutazione delle Prestazioni e Analisi Architetturale 45
3.1 Analisi di Sensitività alla Latenza Crittografica 47
3.1.1 Modellazione del paradigma CTR Mode 47
3.1.2 Modellazione del paradigma ECB Mode 49
3.2 Impatto Topologico: Arità e Profondità dell'Albero . . 52
3.2.1 Setup Sperimentale 53
3.2.2 Analisi della Sensitività all'Arity 53
3.3 Strategie di Evizione ed Impatto Prestazionale 56
3.3.1 Policy 0: Strict Policy 57
3.3.2 Policy 1: Baseline 58
3.3.3 Policy 2: TEC-Tree-Optimized 59
3.3.4 Analisi Comparativa dei risultati 61
3.4 TEC-Tree a confronto con TARDIS-TSO e MESI . . . 64

4 Conclusioni e Sviluppi futuri 67
4.1 Conclusioni 67
4.2 Sviluppi Futuri 69

Capitolo 1

Fondamenti Teorici

1.1 Contesto e Motivazioni

Negli ultimi anni, con la crescente diffusione di applicazioni che gestiscono dati sensibili (dal cloud computing ai dispositivi embedded), la sicurezza dell'hardware è diventata un requisito imprescindibile. Nei modelli di minaccia standard per i sistemi informatici, si assume tipicamente che il perimetro di sicurezza (Trusted Computing Base) sia circoscritto al solo chip del processore. Tutto ciò che risiede all'esterno, compresa la memoria principale off-chip e i bus di interconnessione, è intrinsecamente esposto e vulnerabile. Un attaccante che ha accesso fisico al dispositivo potrebbe leggerne e manipolarne il contenuto. Nell'ambito della sicurezza hardware infatti, i possibili attacchi sono divisi in due categorie:

- **Attacchi Passivi** (Snooping o Bus Probing): l'attaccante si

limita principalmente ad "origliare" il bus leggendo informazioni in chiaro in transito da e verso la memoria, violandone la confidenzialità;

- **Attacchi Attivi**: l' attaccante non si limita a leggere i dati, ma ne altera il contenuto, violandone l'integrità. Tra gli attacchi attivi figurano:
  - **Spoofing**: Iniezione di dati casuali o artefatti al posto di quelli reali;
  - **Splicing (o Relocation)**: L'attaccante prende un blocco di dati valido da un indirizzo di memoria e lo copia in un altro indirizzo;
  - **Replay Attack**: L'attaccante registra un blocco di dati valido e lo reinserisce nella memoria in un secondo momento, riportando la memoria a uno stato pregresso o obsoleto.

Sebbene la cifratura di dati in uscita dall'ultimo livello di cache sia sufficiente a garantire la confidenzialità contro attacchi passivi, essa risulta del tutto inefficace contro attacchi attivi. Un soggetto malintenzionato può sfruttare le tre tecniche precedentemente elencate per alterare il normale flusso di esecuzione di un programma e corrompere i dati, senza che i core possano rilevarne l'intrusione. L'unico metodo in grado di garantire confidenzialità ma allo stesso tempo anche in-

tegrità è una verifica della *freshness* del dato ovvero l'autenticazione della memoria.

1.2 Autenticazione della Memoria

Per contrastare gli attacchi attivi descritti in precedenza, l'approccio base prevede l' associazione di un *Message Authentication Code* (MAC) cioè un'etichetta crittografica calcolata a partire dai dati in uscita dalla LLC (Last Level Cache) e da una chiave segreta memorizzata saldamente all'interno del chip. Sebbene l'uso di un MAC, legato all'indirizzo fisico di memoria, impedisca gli attacchi di Spoofing e Splicing, non è sufficiente a scongiurare i Replay Attack. Un avversario può infatti intercettare una coppia valida composta da "vecchio dato" e "vecchio MAC", per poi sovrascriverla in memoria in un momento successivo: il processore, ricalcolando il MAC sul vecchio dato, lo troverebbe matematicamente corretto, non accorgendosi della manomissione.

1.2.1 Autenticazione basata su Contatori

Per garantire la freschezza del dato e contrastare eventuali attacchi di replay, la crittografia moderna introduce i contatori o indicatori di versione. Ad ogni blocco di memoria è associato un contatore che è incrementato ad ogni operazione di scrittura. Il MAC viene quindi cal-

colato includendo anche questo contatore. Tuttavia, poiché lo spazio on-chip è estremamente limitato, i contatori stessi devono essere memorizzati nella RAM off-chip, esponendo anch'essi al rischio di Replay Attack. La soluzione accademica classica a questo problema è il **Bonsai Merkle Tree (BMT)** che organizza i contatori in una struttura ad albero n-ario:

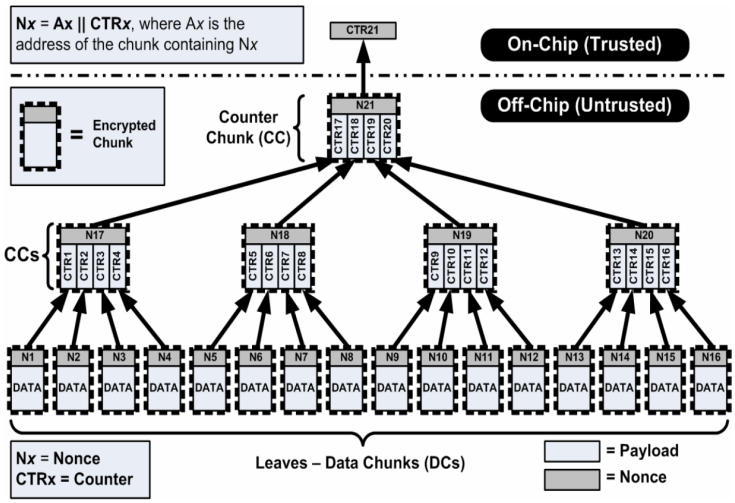
- I contatori dei blocchi dati costituiscono le "foglie" dell'albero.
- Ogni nodo intermedio dell'albero è un MAC che protegge l'integrità dei suoi nodi figli.
- La radice dell'albero (*Root*) è l'unico elemento mantenuto permanentemente al sicuro all'interno del processore (on-chip).

In un BMT, quando si legge un blocco di memoria, il processore deve recuperare il suo contatore e verificare l'intera catena di nodi intermedi fino alla radice. Sebbene tale schema offra le massime garanzie di sicurezza, soffre di un profondo limite prestazionale. La verifica del dato infatti, necessita di numerose letture sequenziali in memoria nonchè di onerosi e ripetuti calcoli crittografici, introducendo un overhead di latenza e un consumo di banda spesso inaccettabili per le moderne architetture ad alte prestazioni.

1.2.2 Tamper Evident Counter Tree

Per superare il collo di bottiglia temporale del BMT lo stato dell'arte si è evoluto verso strutture crittografiche alternative ad alte prestazioni, tra cui spicca il **Tamper-Evident Counter Tree (TEC-Tree)**. A differenza dei tradizionali alberi di hash, il TEC-Tree adotta un paradigma basato sulla cifratura a blocchi unita alla tecnica *Block-Level AREA* (Added Redundancy Explicit Authentication). In questa struttura, i nodi intermedi dell'albero non sono costituiti da digest di hash calcolati sui livelli inferiori, bensì da *Counter Chunk* (CC), ovvero blocchi di memoria contenenti esclusivamente contatori di versione. I contatori delle foglie proteggono i blocchi dati effettivi (*Data Chunk*), mentre i contatori dei nodi intermedi proteggono i nodi figli, fino ad arrivare alla radice (*Root Counter*) conservata on-chip in modo sicuro. Per garantire sia l'integrità che la confidenzialità, il TEC-Tree concatena a ogni blocco (Dati o Contatori) un *nonce* univoco che è tipicamente composto dall'indirizzo fisico del blocco stesso e dal valore del suo contatore genitore. L'intero aggregato (Payload | Nonce) viene poi cifrato utilizzando un cifrario a blocchi in modalità ECB (*Electronic Code Book*) e scritto in memoria. La cifratura in modalità ECB opera su blocchi atomici e indipendenti, tale approccio permette di rendere il processo di cifratura dei diversi blocchi un processo parallelo, a patto che i blocchi dell' albero, dalla foglia alla radice, siano stati gia prelevati dalla memoria. L'autenticazione avviene semplicemente

confrontando, post-decrittazione, il nonce ricostruito dal testo in chiaro con il nonce atteso. L'hardware non è più costretto ad attendere la validazione di un livello per sbloccare l'accesso al livello superiore: tutte le operazioni di decrittazione avvengono simultaneamente. Questa totale assenza di dipendenze sequenziali nel *Critical Path* abbatte drasticamente le latenze, offrendo al contempo confidenzialità "gratuita" (intrinseca alla cifratura) e rendendo il TEC-Tree l'architettura crittografica ottimale per i moderni sistemi multicore.



.

1.3 Il Protocollo di Coerenza Tardis-TSO

In questa sezione si introduce il protocollo di coerenza della cache Tardis-TSO, scelto come base architetturale per l'implementazione del sistema di autenticazione della memoria proposto in questo lavoro. È

importante precisare che il design originale del protocollo Tardis è stato proposto in letteratura da Yu et al,[6] mentre l'implementazione base di Tardis-TSO all'interno del simulatore Gem5, utilizzata come punto di partenza per questo elaborato, è frutto di lavori precedenti.[1] Il contributo originale di questa tesi consisterà, come vedremo nel Capitolo 2, nell'estensione e modifica profonda di tale architettura di base per supportare l'albero crittografico TEC-Tree.

1.3.1 Il modello TSO (Total Store Order)

Nei moderni sistemi multicore, la coerenza della cache rappresenta una sfida significativa sia in termini di prestazione che di traffico sul bus. I protocolli tradizionali, come MESI o modelli *directory-based* standard, prevedono molteplici scambi di messaggi di invalidazione e di *acknowledgment* per garantire che i core osservino una memoria coerente. Tardis è un protocollo di coerenza altamente scalabile che elimina del tutto i messaggi di invalidazione. Nella sua evoluzione, **Tardis 2.0** (o TardisTSO), il protocollo è stato riadattato per supportare il modello di consistenza *Total Store Order*, tipico delle moderne architetture x86.

1.3.2 Il meccanismo dei Timestamp logici

A differenza dei protocolli tradizionali, Tardis-TSO gestisce la coerenza attraverso l'utilizzo di **Timestamp Logici**. L'ordinamento delle

operazioni di memoria non è garantito da broadcast di rete, bensì dall'assegnazione di contatori logici che definiscono l'ordine globale degli accessi. Per supportare il modello TSO, il protocollo suddivide il tempo logico in due componenti principali associate al processore:

- **Load Timestamp (lts)**: il timestamp dell'ultima operazione di Load eseguita;
- **Store Timestamp (sts)**: il timestamp dell'ultima operazione di Store eseguita.

Ciascuna linea di cache presenta inoltre due contatori ad essa associati;

- **Write Timestamp (wts)**: indica la versione del blocco;
- **Read Timestamp (rts) o Lease Time**: rappresenta una scadenza , ovvero l' intervallo di tempo logico entro il quale il core può leggere liberamente la linea di cache.

Durante l'esecuzione, il Load Timestamp del processore cresce. Quando tale valore supera il lease time (rts) associato al blocco di cache, la linea viene considerata logicamente scaduta e il processore è costretto a richiederne il rinnovo.

1.3.3 Architettura del Protocollo

L'architettura Tardis-TSO si articola su due entità principlai che comunicano in modalità *split-transaction*: le cache private di primo livello

(L1) e la Directory centralizzata, che assume il ruolo di *Timestamp Manager*. In particolare:

- **Cache L1**: mantengono i blocchi di dati e i relativi timestamp locali (wts e rts)e possono possedere i blocchi in stato condiviso (*Shared*) per la lettura o in stato esclusivo (*Exclusive*) per la scrittura. Quando un blocco scade, la cache L1 invia una richiesta mirata alla Directory per ottenere un prolungamento del *lease time*.

- **Directory (Timestamp Manager)**: non memorizza esplicitamente i dati ma tiene traccia dello stato e del proprietario di ogni linea di cache, gestendone i timestamp globali. La Directory risolve i conflitti, instrada i blocchi e fornisce i rinnovi del *lease time*, fungendo da arbitro centrale.

1.3.4 Integrazione dell'Autenticazione e Coerenza

Sebbene il TEC-Tree, risolva brillantemente il problema della latenza computazionale parallelizzando le decrittazioni, introduce inevitabilmente un overhead spaziale e di banda. Per ogni blocco dati richiesto, il processore è costretto a recuperare dalla lenta memoria esterna l'intera catena di *Counter Chunk* fino alla radice. Questo processo, se eseguito in modo ingenuo ad ogni operazione di memoria, saturerebbe rapidamente la larghezza di banda del bus, vanificando i vantaggi

prestazionali ottenuti dalla parallelizzazione. Per mitigare questo fenomeno e rendere l'architettura realmente scalabile, si rende necessaria l'introduzione di una **Last Level Cache (LLC)** centralizzata e direttamente accoppiata al nodo *Directory*. L'obiettivo di questa cache dedicata è memorizzare i blocchi dati e i nodi del TEC-Tree più frequentemente utilizzati. Poiché i rami superiori dell'albero (vicini alla radice) vengono condivisi da migliaia di foglie sottostanti, presentano un'altissima località spaziale e temporale. L'inserimento di una LLC permette quindi di "assorbire" la maggior parte delle richieste di autenticazione: se un *Counter Chunk* genitore è già presente in cache, il processore non deve accedere alla DRAM off-chip per recuperarlo. Questa scelta architetturale funge da ammortizzatore per il traffico di rete, permettendo di ispezionare i metadati crittografici quasi istantaneamente e riducendo al minimo gli accessi onerosi alla memoria principale. L'implementazione concreta di questa specifica ottimizzazione architetturale, ovvero l'integrazione del TEC-Tree supportato da una LLC centralizzata all'interno di un protocollo di coerenza, costituisce l'oggetto centrale e il contributo principale della presente tesi. Per modellare, simulare e validare una gerarchia di memoria così complessa, si è reso necessario l'utilizzo di un ambiente di simulazione architetturale avanzato.

1.4 Ambiente di Simulazione GEM5

Il simulatore gem5 è una piattaforma modulare basata su eventi discreti utilizzata nell'ambito dell'architettura dei calcolatori. La composizione modulare del sistema offre la possibilità di riconfigurare, estendere o sostituire ciascun componente dell'architettura adattandosi a molteplici esigenze di simulazione. Tale tool permette di emulare diversi sistemi di calcolo, permettendo la simulazione di diverse architetture come *Alpha*, *ARM*, *MIPS*, *Power*, *SPARC*, *RISC-V* e *x86* a 64 bit. Il simulatore è scritto principalmente in linguaggio C++ e Python e offre spiccata flessibilità nel costruire sistemi di memoria grazie a cache e crossbar ma soprattutto grazie al modulo Ruby, utile per una modellazione avanzata. Le modalità di simulazione previste da gem5 sono principalmente due: **full system mode**, simulando dispositivi e sistemi operativi completi o **syscall emulation** per simulazioni circoscritte nello spazio utente.

1.4.1 Modulo Ruby

Il nucleo centrale di questa ricerca si focalizza sull'impiego del modulo di simulazione Ruby, uno strumento progettato per simulare in modo estremamente accurato il comportamento delle interconnessioni e dei sottosistemi di memoria nei processori moderni. Grazie alla sua spiccata flessibilità nel modellare le gerarchie di memoria e i relativi protocolli, Ruby costituisce l'ambiente ideale per la progettazione e il testing

di nuove soluzioni. In questo contesto, il modulo verrà impiegato per concretizzare il protocollo di sicurezza ed integrità proposto.

1.5 Obbiettivi e Workflow della Tesi

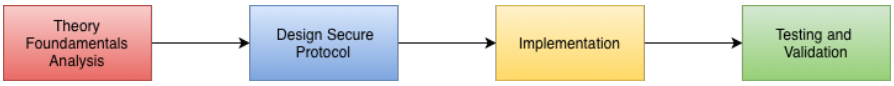


Figura 1.1: Workflow del processo

Il workflow è articolato in quattro fasi e prevede:

1. **Analisi dei Fondamenti Teorici su TEC-TREE**: in questa fase verranno approfondite le specifiche del protocollo di Autenticazione della memoria.

2. **Progettazione del Protocollo**: sulla base dei fondamenti teorici si procederà con la progettazzione del protocollo mediante la definizione di una macchina a stati finiti, messaggi ed eventi.

3. **Implementazione del Protocollo**: gli elementi progettati nella fase precedente saranno tradotti nella sintassi del linguaggio del simulatore gem5 e opportunamente integrati con il protcollo di coerenza preesistente.

4. **Validazione e Analisi Prestazionale**: si procederà alla modellazione e configurazione di un'architettura multicore all'interno dell'ambiente di simulazione, al fine di eseguire benchmark multithread. Tali scenari di test permetteranno di analizzare differenti casi d'uso e di condurre uno studio quantitativo sull'impatto prestazionale, sull'efficacia e sulla scalabilità del protocollo di sicurezza sviluppato.

Capitolo 2

Implementazione del Protocollo Sicuro

In questo capitolo saranno illustrati in maniera estesa i dettagli progettuali e implementativi relativi all'integrazione del protocollo crittografico TEC-Tree all'interno dell'infrastruttura di coerenza Tardis-TSO. A valle dell'analisi teorica condotta nel capitolo precedente, si procederà ora alla descrizione dell'implementazione pratica del sistema all'interno del simulatore architetturale Gem5. Nelle sezioni che seguono verrano presentate ed analizzate le sfide architetturali affrontate per garantire la coesistenza delle logiche di coerenza della cache con i severi requisiti di sicurezza imposti dall'autenticazione della memoria. Particolare attenzione verrà posta sulla riprogettazione degli automi a stati finiti che regolano i controller hardware, evidenziando le differenze sostanziali tra la macchina a stati finiti originale e la nuova architettura

sicura. Infine, verrà presentata la traduzione di tali modelli teorici nello sviluppo di codice a basso livello tramite l'utilizzo del linguaggio SLICC (Specification Language For Implementing Cache Coherence), analizzando l'estensione delle strutture dati, la definizione di nuovi eventi e la stesura di nuove transizioni di stato. Queste operazioni si sono rivelate fondamentali per plasmare e determinare le nuove regole del protocollo TARDISTSO-TECTREE, permettendo al contempo il corretto instradamento dei blocchi crittografici attraverso la rete di interconnessione.

2.1 Modellazione Fisica dell'Integrazione

Il primo e più critico snodo decisionale nello sviluppo di un'architettura di memoria sicura consiste nel determinare in quale punto esatto della gerarchia inserire i controlli crittografici. Integrare il protocollo *Tamper-Evident-Counter Tree* (TEC-Tree) all'interno di un sistema multicore richiede un'attenta riprogettazione dei flussi di dati e di controllo, al fine di garantire l'autenticità delle informazioni senza degradare irrimediabilmente i tempi di risposta della memoria. A differenza dei protocolli di coerenza tradizionali basati su invalidazione esplicita (come le varianti del MESI), il protocollo Tardis-TSO è intrinsecamente governato dalla distribuzione e dal controllo di *timestamp* logici globali,[1] pertanto, piuttosto che demandare l'onere del calcolo crittografico e la verifica di integrità ai singoli nodi di calcolo (ovve-

ro alle cache di primi livello private di ciascun core), si è optato per una soluzione architetturale che centralizza l'intera catena di autenticazione all'interno della Directory stessa. Tale scelta progettuale è maturata a valle della valutazione di molteplici soluzioni architetturali e di uno studio approfondito della letteratura scientifica riguardante l'implementazione di memorie sicure e il loro impatto sulle performance globali del sistema. Come evidenziato dagli autori originali del TEC-Tree nell'analisi dei colli di bottiglia tipici delle architetture di autenticazione basata su alberi,[2] la sfida principale nell'autenticazione della memoria consiste nel minimizzare l'immenso overhead spaziale on-chip mantenendo all'interno dell *Trusted Computing Base* (il confine di fiducia del processore) esclusivamente le radici dell'albero.
Traducendo questo vincolo teorico in un'architettura multicore reale e distribuita, la Directory emerge naturalmente come punto di convergenza ottimale. Questa configurazione trova forte riscontro nella letteratura scientifica: come dimostrato da Gassend, Suh, Declarke, Marten, Devadas,[3] posizionare il motore di verifica crittografica ai livelli superiori della gerarchia causerebbe un aumento insostenibili del traffico di rete per ogni *cache miss*. Al contrario, interfacciandosi direttamente con il *Memory Controller*, all'ultimo livello della gerarchia on-chip (configurazione adottata anche nelle moderne implementazioni hardware trasparenti[5]), la Directory permette di ammortizzare il traffico di rete generato dal recupero dei metadati. Inoltre, fissare il

perimetro di sicurezza e la Root of Trust a livello della Directory evita la ridondante e costosa ri-autenticazione dei dati per ogni singolo hit nelle cache di primo livello.

D'altronde come emerso nell'analisi delle limitazioni imposte dal protocollo Tardis-TSO[1] già implementato, l'assenza di un *Last Level Cache* (LLC) condivisa espone il sistema a gravi colli di bottiglia, costringendo la Directory a continui ed onerosi accessi alla memoria esterna per il recupero dei dati. Tale limite architetturale diviene insostenibile qualora si introduca un albero crittografico come TEC-Tree. Per ovviare a questa intrinseca debolezza, e seguendo le traiettorie di sviluppo futuro tracciate nell'implementazione di partenza,[1] si è deciso di estendere il *Directory Node* equipaggiandolo con una **Mixed Last Level Cache**, visibile nello schema architetturale (Fig.2.1). L'integrazione di una cache di ultimo livello in cui dati standard e metadati crittografici (i Counter Chunk) coesistono e competono per il medesimo spazio allocato, trova inoltre solida giustificazione nella letteratura di settore. Gli studi pionieristici dell' MIT[3] dimostrano infatti che, nonostante la naturale contesa tra i dati applicativi e l'albero, l' adozione di una LLC mista permette di abbattere il catastrofico overhead prestazionale, tipico delle implementazione crittografiche ingenue, confinandolo entro un margine altamente accettabile.

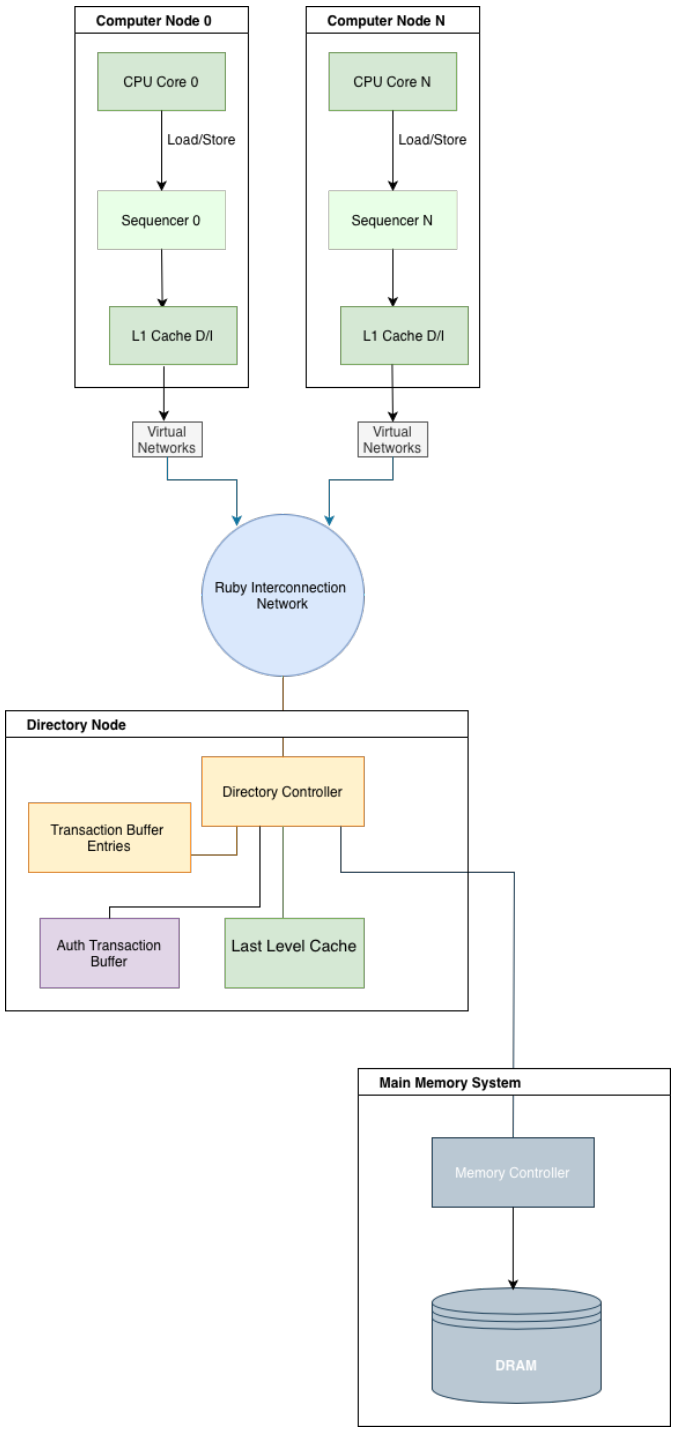


Figura 2.1: Schema Architetturale

2.1.1 Partizionamento dello Spazio Fisico

Dal punto di vista strettamente topologico e di modellazione della memoria principale (*DRAM off-chip*), l'integrazione del TEC-Tree ha imposto una rigorosa suddivisione dello spazio di indirizzamento fisico. Per poter memorizzare l'albero crittografico la DRAM è stata suddivisa in due macro-aree distinte:

- **Data Chunk (DC)**:questa regione comprende i blocchi di memoria convenzionali, i quali ospitano i dati veri e propri (istruzioni e variabili) manipolati dai workload in esecuzione sui processori. Nel contesto dell'albero crittografico, questi blocchi costituiscono le "foglie" terminali;

- **Counter Chunk (CC)**: Questa regione, completamente trasparente al livello applicativo e al sistema operativo, è dedicata allo stoccaggio esclusivo dei metadati di sicurezza. Essa contiene i blocchi che ospitano i contatori di versione, i *Nonce* e i nodi intermedi del TEC-Tree. L'accesso a quest'area di memoria è strettamente riservato al controller della Directory, che vi accede per percorrere la struttura ad albero dal nodo foglia fino alla Root, al fine di verificare la *freshness* (freschezza) del dato e sventare attacchi di tipo Replay.

2.1.2 Evoluzione Strutturale della Directory

La nuova Directory progettata, non solo per fornire timestamp coerenti, ma anche per garantire l'onere crittografico, è quindi dotata di una propria cache interna (L2 Cache 2.2), le cui dimensioni sono configurabili a runtime e destinata allo stoccaggio concorrente sia di dati (DC) che di metadati (CC). Questa fondamentale estensione della gerarchia di memoria si affianca ad un'ulteriore separazione netta tra le *Transaction Buffer Entries* (TBE) tradizionali e una nuova struttura dati dedicata, introdotta in questa implementazione e denominata **Authentication Transaction Buffer Entries** (AuthTBE). La necessità di una distinzione tra queste strutture nasce principalmente dalla netta differenza logica tra il recupero del dato e la sua autenticazione.

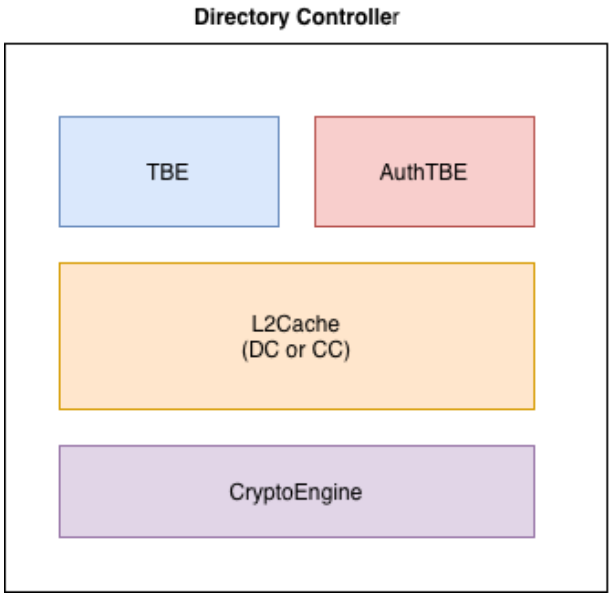


Figura 2.2: Component Diagram Directory

TBE vs AuthTBE

Nel modello Tardis-TSO originale, i tradizionali *TBE (Transaction Buffer Entries)* assolvono al compito di tracciare le transazioni sui blocchi di memoria convenzionali (DC). Essi sono allocati all'arrivo di una richiesta e mantengono in memoria l'indirizzo del dato e il *MachineID* del core richiedente, bloccando ("congelando") la macchina a stati finché la DRAM non restituisce il payload. Una volta servita la richiesta, il TBE è immediatamente deallocato. Tuttavia l'implementazione del protocollo TEC-Tree impone di risalire, durante le autenticazioni di un blocco, l'intero albero fino alla radice, generando nel mentre nuove richieste per i *Counter Chunk* e mantenendo il Data Chunk originale in sospeso. Per evitare situazioni di deadlock strutturali e conflitti di stato è stato definito in linguaggio Slicc una nuova struttura : l'**AuthTBE**. Questo particolare *transaction buffer* agisce come gestore di stato altamente specializzato nelle operazioni di fetch di metadati. Analizzando la sua definizione nel codice sorgente (`TARDISTSO_TECTREE-dir.sm`), è possibile osservare i campi architetturali che lo distinguono nettamente da un TBE tradizionale:

- **OriginalAddress**: L'AuthTBE tiene traccia dell'indirizzo originale (DC) in stallo;
- **ParentAddress**: l'indirizzo del blocco "padre" correntemente richiesto alla memoria. Questo permette al protocollo di calcolare dinamicamente la traiettoria di risalita verso la Root;

- **Waiter Slots:** L'innovazione più significativa dell'AuthTBE risiede nell'implementazione di un array hardware di slot di attesa (Es: da `Waiter0` a `Waiter15`). A causa dell'arietà dell'albero (N-ary tree) infatti, blocchi di dati spazialmente contigui condividono il medesimo blocco padre di metadati. Se due o più core richiedono (o sfrattano) contemporaneamente dati vicini, la Directory potrebbe generare molteplici *Counter Miss* identici. L'AuthTBE risolve questa ridondanza: se un fetch per un determinato padre è già stato annotato, le nuove richieste non interrogano la DRAM, ma si mettono "in coda" occupando i campi *Waiter* dell'Entry già esistente. Quando il blocco padre viene infine recuperato dalla memoria, la Directory ispeziona l'AuthTBE e risveglia simultaneamente tutte le transazioni accodate (fino alla massima arietà supportata), autenticandole in parallelo. Questa meccanismo garantisce una riduzione di traffico ridondante sul bus di sistema ed evita i meccanismi di *coelescing* introdotti da Ruby per la gestione di richieste per gli stessi indirizzi verso il controller di memoria.

2.2 Modifica dell'Automa in SLICC

Le modifiche strutturali apportate al nodo Directory impongono una completa riprogettazione dell'automa a stati che regola il *Directory controller*. L'introduzione di sicurezza crittografica infatti, costringe la Directory a non rilasciare il dato richiesto in lettura al processore se prima non sia stata ultimata la verifica dell'intero ramo dell'albero crittografico (*Autentication*). Allo stesso modo le operazioni di scrittura non possono considerarsi concluse finchè non sono stati aggiornati tutti i Counter Chunk del percorso che va dalla foglia (DC) fino alla radice (*Secure Write*). Per modellare questo comportamento asincrono è stato necessario estendere le regole del protocollo orginale. Nei sottoparagrafi che seguono saranno analizzate in dettaglio tutte le novità introdotte nel flusso del codice SLICC. Si procederà dapprima con la definizione dei nuovi stati architetturali e dei nuovi eventi generati dal sistema, per poi illustrare, tramite i relativi diagrammi di transizione, la nuova FSM che gestisce lo scambio di dati e metadati tra le la Directory e gli altri nodi della gerarchia di memoria (Cache L1 e DRAM).

2.2.1 Nuovi Stati Architetturali

Per ottenere la coesistenza di dati applicativi e metadati all'interno della medesima struttura, l'automa a stati finiti della Directory è stato significativamente esteso. Oltre agli stati previsti dal protocollo

Tardis-TSO[1] (Invalid, Shared, Exclusive e i relativi transitori) sono stati definiti:

- **I_Evicting**: stato transitorio in cui si trova un blocco dati (precedentemente in stato *Invalid*) per i core, ma ritenuto *Dirty* in LLC in fase di evizione. Il blocco è in attesa di ricevere un *Memory Ack* dal controller della DRAM a conferma dell'avvenuta scrittura;

- **S_Evicting**: simile al percedente , rappresenta lo stato di un blocco originariamente in stato *Shared* che è stato selezionato per l'evizione dalla LLC verso la memoria principale;

- **E_Evicting**: stato trasitorio per l'evizione di un blocco che si trovava in stato *Exclusive*. Anche in questo caso, la Directory attende il completamento della transazione sul bus di memoria;

- **I_Fetch_Auth**: stato transitorio critico per la sicurezza. Un blocco (richiesto da stato *Invalid*) è stato recuperato dalla DRAM o dalla LLC, ma non può ancora essere consegnato al processore. La macchina a stati è "congelata" in attesa che il rispettivo blocco dei contatori (*Counter Chunk*) venga prelevato dalla memoria per verificarne l'integrità;

- **S_Fetch_Auth**: analogo al precedente, rappresenta uno stallo per l'autenticazione crittografica di un dato richiesto con permessi di lettura (*Shared*);

- **E_Fetch_Auth**: stato di attesa per l'autenticazione di un blocco richiesto con permessi di scrittura (*Exclusive*);

- **C_M**: stato stabile (*dirty*) assegnato ai blocchi di metadati in LLC che sono stati aggiornati a seguito di una scrittura in memoria di un blocco dati al quale il Counter Chunk è associato. Un blocco in `C_M` dovrà obbligatoriamente essere scritto in memoria principale qualora venga sfrattato dalla cache;

- **C_V**:stato stabile (*clean*) assegnato ai blocchi di metadati (*Counter Chunk*) residenti all'interno della mixed LLC;

- **E_Evict_Auth**: stato transitorio per uno sfratto congelato derivante da un blocco in stato *Exclusive*, in attesa del recupero dei metadati crittografici.

- **I_Evict_Auth**: si innesca quando la LLC tenta di evincere un blocco dati *Dirty*, ma il relativo blocco dei contatori non è presente in cache (*Counter Miss*);

- **S_Evict_Auth**: stato transitorio per uno sfratto congelato derivante da un blocco in stato *Shared*, in attesa del recupero dei metadati crittografici;

- **E_Evict_Flush**: stato transitorio speciale. Si verifica quando un blocco in stato *Exclusive* deve essere evinto dalla LLC, ma la Directory deve prima attendere che il core proprietario (L1)

invii il blocco modificato tramite un messaggio di *FlushRep*, prima di poter procedere all'eventuale autenticazione e scrittura in DRAM.

2.2.2 Estensione degli Eventi

Per poter governare le transizioni verso i nuovi stati, l'automa della Directory è stato istruito a riconoscere una nuova gamma di eventi. Nel linguaggio SLICC, un evento (*Event*) rappresenta lo stimolo che induce la macchina a stati a intraprendere una nuova transizione, modificando il nuovo stato e producendo eventuali messaggi di risposta. Mentre il protocollo originale reagiva alle richieste dei core a alle risposte della DRAM, il protocollo TARDIS TEC-TREE introduce una complessa rete di eventi auto-generati, necessari per la gestione autonoma dell' ultimo livello di cache e per gestire i processi di *Counter Fetch*. Tra i principali eventi integrati nel sistema figurano:

- **ShReq_Hit** ed **ExReq_Hit**: rappresentano richieste di lettura (*Shared*) o scrittura (*Exclusive*) provenienti da una cache di primo livello (L1) che risultano in *Hit* all'interno della LLC della Directory. A differenza delle richieste standard, questi eventi permettono di bypassare l'accesso alla memoria principale, abbattendo la latenza di recupero del dato;

- **Counter_Data**: evento scatenato dall'arrivo di un blocco Counter Chunk dalla memoria DRAM esterna;

- **LLC_Repl_Clean**: trigger interno generato dalla Directory quando la LLC risulta piena ed è necessario fare spazio per un nuovo blocco. Questo evento interno permette l'evizione senza generare nuove richieste di writeback in memoria (*silent eviction*).

- **LLC_Repl_Dirty**: si tratta di un trigger interno di rimpiazzamento di un blocco *Dirty* che a differenza del precedente impone che il blocco evinto venga immediatamente riscritto in memoria DRAM.

- **Auth_Miss**: evento di vitale importanza per la sicurezza del sistema. Si verifica quando un Data Chunk è pronto per essere elaborato, ma il blocco di metadati ad esso associato (necessario per verificarne l'integrità) non è presente in LLC. Questo evento forza la Directory ad allocare un AuthTBE e a richiedere i contatori alla DRAM;

- **LLC_Repl_Dirty_Auth_Miss**: evento scatenato durante il tentativo di evizione di un blocco Dirty, quando si scopre che i metadati di sicurezza necessari per autorizzarne l'aggiornamento in memoria non sono presenti in cache. Induce il passaggio verso gli stati di "sfratto congelato" (es. *E_Evict_Auth*);

- **LLC_Repl_Dirty_Auth_Hit_From_Flush**: evento di trigger interno che si attiva dopo la ricezione di un messaggio di

FlushRep (ovvero il writeback volontario di un dato da parte di una L1 verso la Directory). Questo evento specifico indica che i contatori necessari per processare il Flush sono già disponibili (Hit) all'interno della LLC.

- **LLC_Repl_Dirty_Auth_Miss_From_Flush**: analogamente al precedente, questo evento si attiva dopo un *FlushRep*, ma segnala alla macchina a stati che i metadati associati al blocco in fase di Flush sono assenti in LLC (Miss), obbligando la Directory a sospendere il writeback e a innescare il recupero dei contatori.

2.2.3 Automa a Stati Finiti: Data Chunk

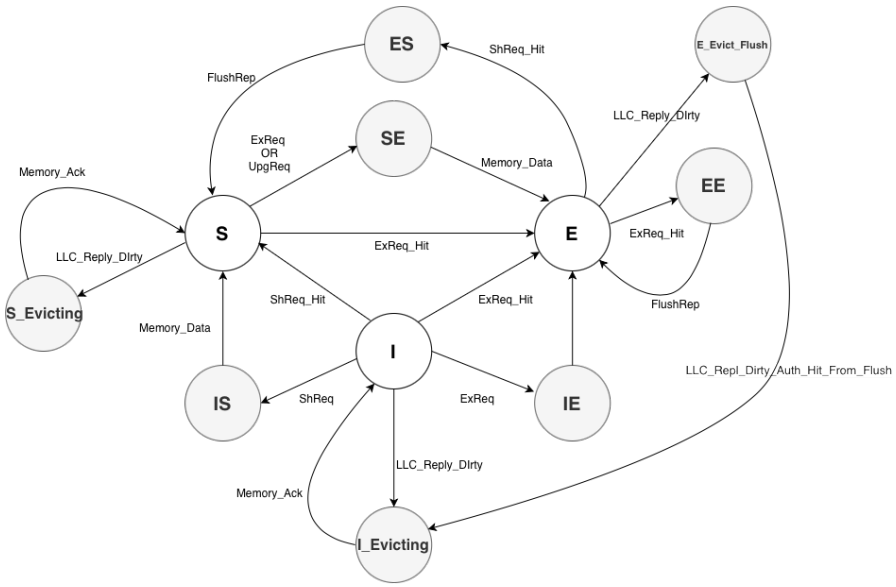


Figura 2.3: Automa a stati finiti per i Data Chunk

Nel protocollo Tardis originale, il percorso di un blocco dati è strettamente legato alla sua disponibilità nelle cache di primo livello: in caso di miss, la Directory interrogava la memoria DRAM e, alla ricezione del dato, transitava immediatamente allo stato stabile *Shared* o *Exclusive*, inoltrando il payload al processore richiedente con nuovo timestamp. Tuttavia, l'implementazione della sicurezza basata su TEC-Tree stravolge tale paradigma. Come illustrato in figura 2.3, il rilascio di un blocco dati è subordinato alla sua presenza fisica, alla presenza dei blocchi Counter a esso associati e al conseguente superamento dei controlli d'integrità. Analizzando i flussi d'esecuzione emergono quindi due percorsi architetturali ben distinti:

- *Hit* (Fast-Path): se un core richiede un dato che è gia valido

all'interno della mixed LLC, la directory genera eventi di *ShReq_Hit* o *ExReq_Hit*. In questo caso, il controllo di sicurezza non è necessario, poichè il dato si trova già all'interno della zona *Trusted*, pertanto la linea di cache richiesta transita direttamente negli stati stabili (S o E);

- *Miss* (Stallo Crittografico): nel caso in cui il blocco dati richiesto non sia presente in cache L2, si innesca il lookup per i metadati. Piuttosto che transitare verso gli stati stabili, la coda forza il dato in uno stato di "stallo transitorio" (identificati dai nodi grigi degli schemi 2.4 e 2.5). Durante questa fase, la richiesta per il dato viene congelata e la directory si occupa dell'allocazione degli AuthTBE necessari per il recupero ricorsivo dei Counter Chunk mancanti.

Lo stato di stallo per il dato si protrae fin quando non è risolto l'intero ramo crittografico. Solamente alla ricezione dell'ultimo *Counter_Data* mancante e a valle della sua decifratura, la Directory genera un evento interno di *WakeUp*. Tale evento risveglia la richiesta originale per il dato, facendo evolvere l'automa verso lo stato stabile. Tali dinamiche si verificano sia durante le normali richieste dei core, sia durante i processi di evizione dei *Data Chunk*. È fondamentale sottolineare che, al termine di questo processo, i *Counter Chunk* del ramo vengono marcati come *Modified* (stato C_M) unicamente nel caso di sfratti di blocchi "sporchi"; nelle semplici operazioni di lettura (*ShReq*), invece,

essi mantengono inalterato il loro stato originario. Le dinamiche di stallo fin qui descritte evidenziano come il flusso dei dati applicativi sia totalmente subordinato alla disponibilità dei metadati. Diviene pertanto fondamentale analizzare il modo in cui la Directory gestisce il ciclo di vita, il rimpiazzamento e gli aggiornamenti di questi ultimi.

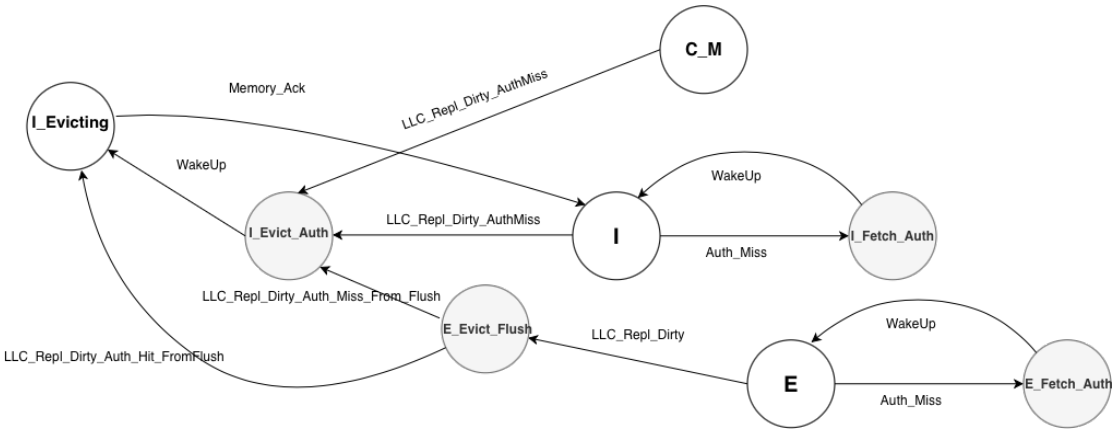


Figura 2.4: Processo di Autenticazione

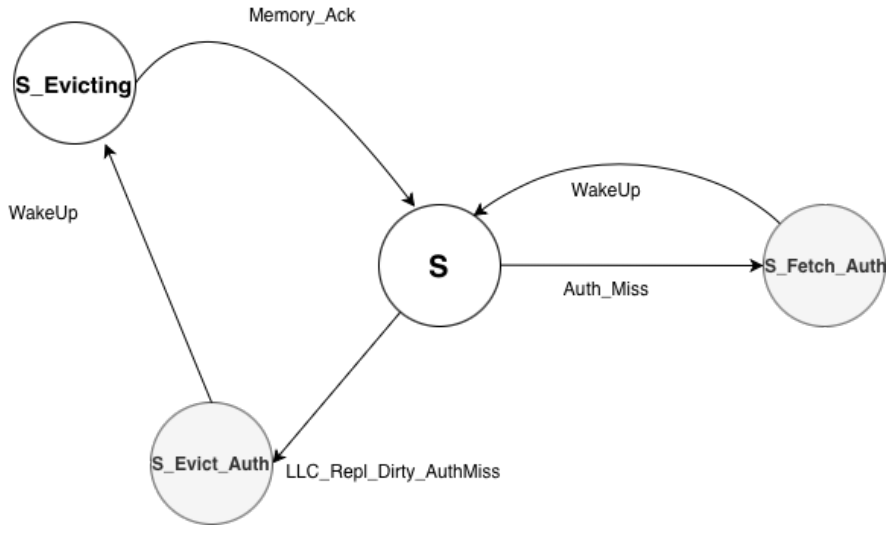


Figura 2.5: Processo di Autenticazione

2.2.4 Automa a Stati Finiti: Counter chunk

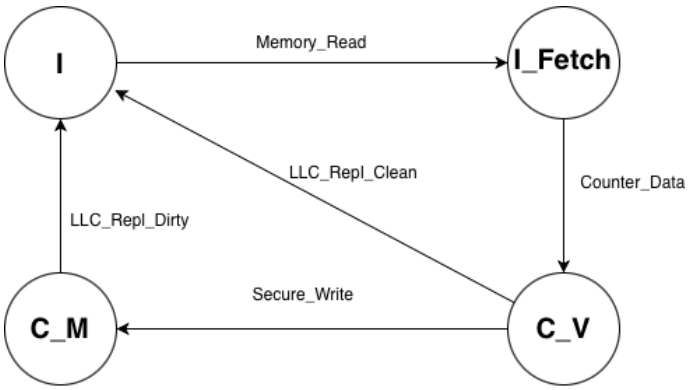


Figura 2.6: ciclo di vita di un Counter Chunk

A differenza dei blocchi dati convenzionali, i *Counter Chunk* non vengono mai richiesti direttamente dai processori, ma sono richiesti dalla Directory per soddisfare i controlli di integrità a seguito di un *Auth_Miss*. Per gestire autonomamente questa porzione "invisibile" ai core dello spazio d'indirizzamento, l'automa a stati finiti adotta un modello dedicato, mostrato in figura 2.6. Un blocco di metadati attraversa quindi quattro differenti stati:

- *Invalid*: il Counter Chunk non è presente in L2;
- *I_Fetch*: stato transitorio in cui la linea transita a seguito di una richiesta alla DRAM per quel blocco contatori;
- *C_V*: l'arrivo del Counter Chunk dalla memoria (evento *Counter_Data*) permette l'allocazione del blocco in L2, il blocco è considerato *Clean* (identico a quello in memoria). Qualora do-

vesse essere sfrattato per fare spazio, subirebbe un'evizione silenziosa tornando in stato *Invalid*.

- C_M: quando un blocco counter viene modificato a seguito di una *Secure Write*, il blocco diventa *Dirty*, ciò significa che una potenziale evizione futura imporrà la riscritttura in memoria (*writeback*).

2.2.5 Riepilogo delle Transizioni dell'Automa SLICC

A valle della definizione dei nuovi stati e dei nuovi eventi crittografici, è possibile formalizzare l'intero comportamento del Directory Controller. Per ragioni di chiarezza espositiva, le transizioni che compongono il nuovo automa SLICC sono state suddivise in tre macrolocazioni logiche, permettendo di isolare il normale traffico di coerenza dalle rigorose politiche di sicurezza e di rimpiazzamento. Nella Tabella 2.1 sono riassunte le transizioni di base ereditate dal protocollo Tardis-TSO, responsabili del normale recupero dei dati dalla memoria.

Stato Iniziale	**Evento (Richiesta)**	**Stato Finale**
I	ShReq	IS
I	ShReq_Hit	S
I	ExReq	IE
I	ExReq_Hit	E
S	ShReq	IS
S	ExReq	SE
S	UpgReq	E
E	ShReq, ShReq_Hit	ES
E	ExReq, ExReq_Hit, UpgReq	EE
E, EE	FlushRep	S, E
ES	FlushRep	S
IS, IE	Memory_Data	S, E
SE	Memory_Data	E
{IS, IE, SE, ES, EE}	{ShReq, ShReq_Hit, ExReq, ExReq_Hit, UpgReq}	*Stallo (Retry)*

Tabella 2.1: Automa Directory: Transizioni di Base

L'integrazione del TECTREE stravolge questo flusso introducendo gli stati di attesa crittografica. Come mostrato nella Tabella 2.2, ogniqualvolta si verifica un *Auth_Miss*, l'automa blocca la transazione corrente e la parcheggia in stati transitori (es. *I_Fetch_Auth*), mettendo parallelamente in stallo tutte le successive richieste di coerenza che collidono con il blocco in fase di autenticazione, al fine di prevenire attacchi e inconsistenze.

Stato Iniziale	**Evento Scatenante**	**Stato Finale**
I, S, E	Auth_Miss	I_Fetch_Auth, S_Fetch_Auth, E_Fetch_Auth
I	Counter_Data	I
{I,S,E}_Fetch_Auth	{ShReq, ShReq_Hit, ExReq, ExReq_Hit, FlushRep, UpgReq, Auth_Miss}	*Stallo*
{IS, IE, SE, EE, ES}	Auth_Miss	*Stallo*

Tabella 2.2: Automa Directory: Transizioni di Validazione Crittografica (TEC-Tree)

Infine, la Tabella 2.3 formalizza le rigorose logiche di evizione. Se nei protocolli tradizionali lo sfratto di un blocco sporco richiede una semplice scrittura in DRAM, l'automa TEC-TREE impone una rigida gerarchia: nessun blocco modificato può lasciare la LLC se prima non si è garantita la validità dell'intero ramo dei contatori (*Counter Miss durante Eviction*), portando a sfratti congelati.

Stato Iniziale	**Evento di Sfratto**	**Stato Finale**
I, S	LLC_Repl_Dirty	I_Evicting, S_Evicting
I, S	LLC_Repl_Dirty_ Auth_Miss	I_Evict_Auth, S_Evict_Auth
E	LLC_Repl_Dirty, LLC_Repl_Dirty_ Auth_Miss	E_Evict_Flush
C_M	LLC_Repl_Dirty	I_Evicting
C_M	LLC_Repl_Dirty_ Auth_Miss	I_Evict_Auth
E_Evict_Flush	LLC_Repl_Dirty_ Auth_Hit_From_Flush	I_Evicting
E_Evict_Flush	LLC_Repl_Dirty_ Auth_Miss_From_Flush	I_Evict_Auth
I_Evicting, S_Evicting, E_Evicting	Memory_Ack	I, S, E
{I, S, E}_Evicting	FlushRep, ShReq, ExReq, UpgReq, Auth_Miss	*Stallo*
{I, S, E}_Evict_Auth	LLC_Repl_Clean, ShReq, ExReq, Auth_Miss	*Stallo*
Tutti gli stati transitori e di Fetch/Evict	LLC_Repl_Clean, LLC_Repl_Dirty	*Stallo*

Tabella 2.3: Automa Directory: Transizioni di Rimpiazzo e Scrittura in Memoria (Eviction)

2.3 Interfaccia di Rete e Code SLICC

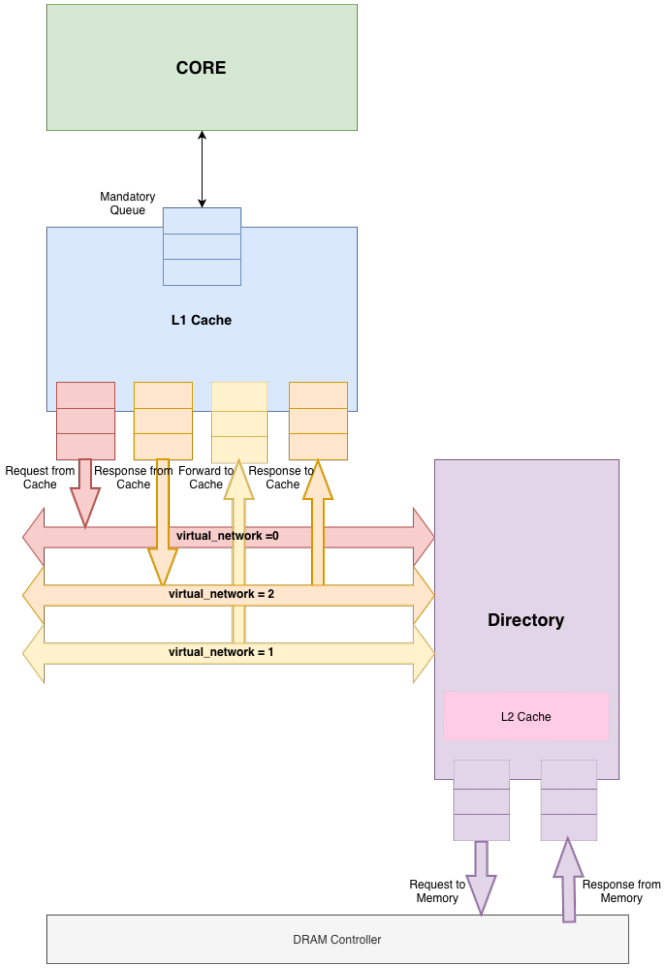


Figura 2.7: Schema d'interconnessioni della Directory

L'architettura del Directory Controller non opera come entità separata, ma funge da fulcro di coordinamento per le transazioni di memoria all'interno del sistema multiprocessore. Per far sì che la coerenza e i requisiti d'integrità possano sussistere, la Directory deve comunicare in modo efficiente oltre che efficace sia con le cache di primo livello (L1 private dei core) che con la DRAM. Come si evince dalla figura 2.7, l'infrastruttura per la comunicazione di rete, ereditata dal pro-

tocollo Tardis originale, si basa sull'impiego di *Message Buffer*: code hardware FIFO (*First-In-First-Out*) per collegare il controller sia alla cache L1 che alla memoria principale. Da un punto di vista strutturale, questi canali mantengono la classica suddivisione in due flussi principali:

- **Inbound (Code in ingresso):** per la ricezione delle richieste dai core (*RequestFromCache*) e delle risposte dalla memoria DRAM (*ResponseFromMemory*).
- **Outbound (Code in uscita):** per l'inoltro di richieste di Flush o writeback verso le L1 (*ForwardToCache*), l'invio dei payload dati e timestamp (*ResponseToCache*) e le richieste verso la DRAM (*RequestToMemory*).

Tuttavia, sebbene la struttura topologica delle code sia rimasta inalterata, il comportamento e la logica di smistamento dei messaggi al loro interno sono stati profondamente riscritti per supportare le rigidità introdotte dal protocollo TEC-Tree.

2.3.1 Inbound Queue

Il flusso dei messaggi in entrata verso la Directory rappresenta lo snodo più critico dell'intera architettura TEC-Tree. È proprio in questa fase, infatti, che il normale traffico di coerenza viene intercettato per garantire l'integrità e l'autenticazione dei blocchi di memoria. L'archi-

tettura fa affidamento su due principali *Message Buffer* in ricezione: la *requestFromCache* e la *responseFromMemory*.

Request From Cache

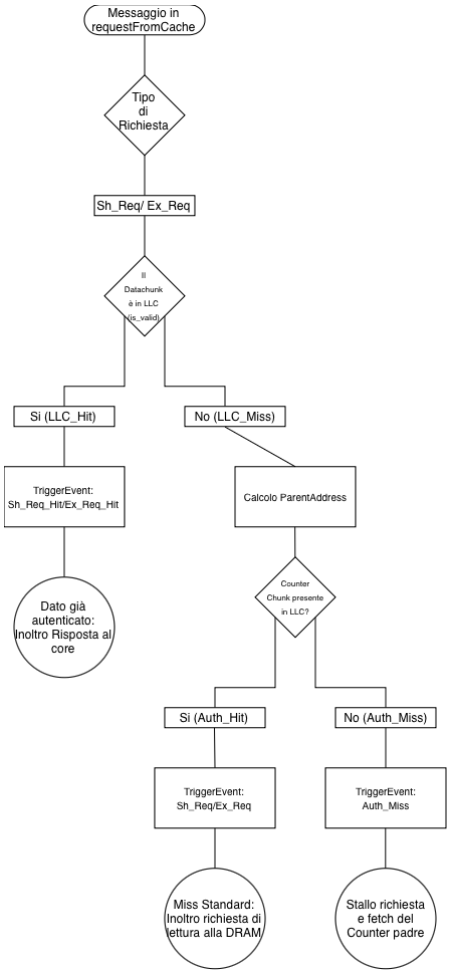


Figura 2.8: Flow Diagram RequestFromCache queue

Come si evince dal Flow Diagram 2.8, ciascun messaggio ricevuto sulla coda di richiesta che collega le L1 alla Directory, viene preliminarmente classificato o come richiesta di lettura da parte di un core (Sh_Req) o come una richiesta di Scrittura (Ex_Req). Il primo bivio logico fondamentale per la coda è stabilire tramite lookup in cache se l'in-

dirizzo richiesto è presente e valido in cache. In caso affermativo, il dato è già all'interno del perimetro sicuro, pertanto non necessita di autenticazione. In caso contrario invece, la coda si occupa, tramite una funzione ausiliaria implementata in *RubySlicc_Util.hh*, di calcolare il suo corrispondente "padre" e verificarne la presenza in LLC. Se tale blocco è gia presente (*Auth_Hit*) allora l'evento scatenato sarà quello di *Sh_Req/Ex_Req* e di conseguente richiesta alla memoria per il dato. In caso invece di mancata presenza del Counter Chunk associato, la richiesta originale sarà sospesa e la coda scatenerà un evento di *Auth_Miss*. Inoltre è utile evidenziare come la coda sia programmata per riconoscere solamente richieste di tipo Shared o Exclusive, pertanto ciascuna messaggio non omologato a questi formati verrà automaticamente scartato dalla coda e classificato come *Unexpected Message*.

Response From Memory

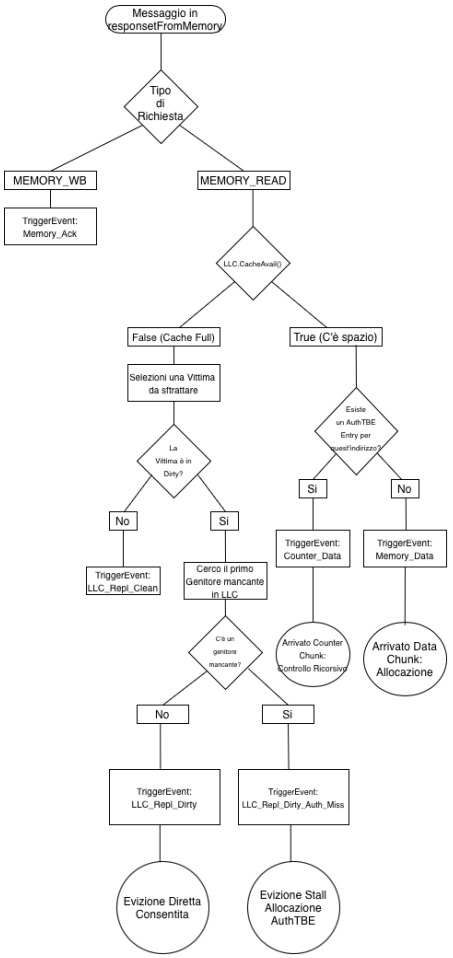


Figura 2.9: Flow Diagram ResponseFromMemory queue

La porta *responseFromMemory* rappresenta il terminale di ricezione per tutti i pacchetti provenienti dalla memoria principale (DRAM). Mentre in un protocollo di coerenza standard questo canale si limita a inoltrare i dati richiesti verso i livelli superiori, nell'architettura TEC-Tree esso assume il ruolo di un crocevia decisionale complesso, fondamentale per garantire l'integrità. Come schematizzato nel diagramma di flusso in Figura 2.9, all'arrivo di un pacchetto di tipo *MEMORY_READ*, il controller valuta in primo luogo la disponibilità di

spazio all'interno della Mixed LLC. Qualora la cache risulti piena (*Cache Full*), si innesca la logica di evizione. È in questa fase che interviene il primo vincolo crittografico del protocollo: se la "vittima" selezionata per lo sfratto è un blocco modificato (*Dirty*), il suo aggiornamento in memoria centrale richiede necessariamente la disponibilità dei suoi *Counter Chunk* associati per ricalcolarne la firma. Di conseguenza, il controllore invoca la funzione di ispezione *getFirstMissingParent()* per scansionare la gerarchia:

- Se un solo genitore è assente, lo sfratto viene bloccato. Viene generato un evento *LLC_Repl_Dirty_Auth_Miss* che sospende l'evizione all'interno di una Transient Buffer Entry dedicata (*AuthTBE*), avviando il fetch del metadato mancante.
- Se tutti i genitori sono presenti (o se il blocco vittima era *Clean*), l'evizione diretta è consentita.

Nel caso in cui, invece, vi sia spazio disponibile in cache (o al termine dell'evizione), il controllore deve discriminare la natura del pacchetto in arrivo, poiché Dati e Contatori condividono il medesimo bus fisico. Il routing avviene verificando la presenza dell'indirizzo del pacchetto all'interno della tabella degli stalli (*AuthTBEs*):

- **Assenza in AuthTBE:** il pacchetto viene identificato come un *Data Chunk* convenzionale. L'evento *Memory_Data* si occupa di allocarlo e di concludere la transazione verso il core.

- **Presenza in AuthTBE:** il pacchetto viene identificato come un *Counter Chunk* precedentemente richiesto a causa di un *Auth Miss*. L'evento *Counter_Data* salva il contatore in LLC e innesca un meccanismo di verifica ricorsiva *bottom-up*. Il sistema controlla se il genitore di questo nuovo contatore sia a sua volta mancante: in caso affermativo la ricorsione prosegue verso l'alto (es. verso il "nonno"); in caso negativo (catena di fiducia ripristinata fino alla root), il controller esegue un risveglio globale, sbloccando la transazione dati originaria.

2.3.2 Outbound Queue

Una volta concluse le transazioni verso gli altri componenti della gerarchia di memoria e le autenticazioni crittografiche la Directory si occupa di instradare i messaggi di risposta verso le corrette interfacce di rete. A tale scopo, l'architettura si affida a tre principali *Message Buffer* in uscita:

- *ForwardToCache*: ha lo scopo di instradare verso la cache private dei core i messaggi di *FLUSH_REQ* o le *WB_REQ*, invalidazioni forzate rispettivamente da aggiornamenti su dati o da cambi di Owner per la linea di cache;

- *ResponseToCache*: tale struttura è utilizzata per rispondere con i dati alle richieste avanzate in precedenza dalle cache;

- *RequestToMemory*: coda utilizzata per instradare le richieste di Data chunk e Counter chunk alla memoria DRAM.

Capitolo 3

Valutazione delle Prestazioni e Analisi Architetturale

L'implementazione di un protocollo di coerenza sicuro e consapevole della presenza di metadati crittografici, come quello descritto nei capitoli precedenti, richiede una rigorosa validazione sperimentale per quantificarne i compromessi architetturali. In questo capitolo viene presentata l'analisi prestazionale del protocollo *TARDIS-TSO TEC-Tree*, condotta attraverso un'ampia gamma di esperimenti ciclo-accurate. L'obiettivo primario di questa valutazione è analizzare l'impatto applicativo introdotto dai meccanismi di protezione della memoria, misurando le variazioni dei tempi di esecuzione in risposta a specifiche

modifiche architetturali. Tutte le sperimentazioni sono state eseguite avvalendosi del simulatore *Gem5* in modalità *System Call-Emulation*, sfruttando il sottosistema di memoria *Ruby* per modellare fedelmente la latenza e il traffico generato dai trasferimenti dei Dati e dei *Counter Chunk* (nodi dell'albero di sicurezza). Come carico di lavoro (*workload*) di riferimento per sollecitare l'architettura è stato scelto l'algoritmo di ordinamento *Radix Sort*, un benchmark ampiamente riconosciuto per la sua elevata intensità di accessi in memoria e per i suoi complessi pattern di condivisione dei dati in ambienti multithreading. Al fine di garantire la massima precisione e ripetibilità degli esperimenti, i risultati estratti dalle simulazioni sono stati rigorosamente confinati alla sola *Region of Interest* (ROI) dell'applicazione. Tale isolamento è stato implementato tramite l'inserimento di specifiche istruzioni di annotazione (magic instructions) nel codice sorgente del benchmark, permettendo al simulatore di ignorare le fisiologiche fasi di inizializzazione e distruzione delle strutture dati e di collezionare le statistiche unicamente durante l'effettiva esecuzione dell'algoritmo parallelo.

3.1 Analisi di Sensitività alla Latenza Crittografica

L'integrazione di motori crittografici all'Interno del *Memory Controller* introduce inevitabilmente un costo temporale all'elaborazione dei dati. Al fine di valutare la resilienza del protocollo di sicurezza e le sue performance nel contesto della gerarchia di memoria è stata condotta un'approfondita analisi di sensitività. L'obbiettivo primario è quantificare l'impatto della crittografia hardware in due scenari architetturali opposti: il paradigma crittografico moderno basato su **AES-CTR** e il paradigma, citato nel TEC-Tree, basato invece su **AES-ECB**.

3.1.1 Modellazione del paradigma CTR Mode

Come descritto da Shay Gueron, nel design del *Memory Encryption Engine* (MEE) per architetture Intel,[4] nei moderni processori protetti, i dati in chiaro non sono cifrati direttamente; infatti l'AES utilizza l'indirizzo fisico del dato e un contatore temporale univoco per generare un *Nonce* pseudocasuale di bit da combinare al dato con un'operazione logica di XOR, circuito combinatorio in grado di effettuare l'elaborazione istantaneamente (a latenza trascurabile). Pertanto la generazione della chiave può avvenire in parallelo all'accesso in DRAM. Nel nostro modello questo si traduce in una *sendMemoryRequest*, *action* Slicc che non subisce alcuna penalità crittografica aggiuntiva e il ritardo applica-

to corrisponde unicamente a quello del controller di memoria (*latency hiding*). Se in questo modello, per i dati, il concetto di latenza nascosta risulta molto vantaggioso in termini di performance, non si può dire la stessa cosa per i metadati per i quali invece, la struttura ad albero introduce una dipendenza strutturale detta Catena di Fiducia.[4] Non è infatti possibile generare il *Keystream* per un nodo figlio senza aver prima decifrato e validato il padre. Per riflettere questa rigorosa barriera sequenziale imposta dall'hardware, nell'azione SLICC dedicata all'esplorazione dell'albero (*checkRecursionAndWakeUp*), è stata deliberatamente iniettata la penalità crittografica completa (*crypto_latency*) per ogni singolo livello esplorato a seguito di *Auth-Miss* del contatore padre.

Per avviare la simulazione di questo paradigma architetturale è stato sviluppato uno script *bash* dedicato che esegue test ripetuti incrementando dinamicamente la latenza crittogafica. Il comando per lanciare la *batch suite* in modalità Counter Mode in GEM5 è il seguente:

```
./4.run_crypto_latency_sweep.sh 0 CTR
```

Dove il primo parametro (`0`) indica la disattivazione temporanea della politica di rimpiazzo MRU per isolare il comportamento crittografico e il secondo parametro (`CTR`) istruisce il simulatore a non penalizzare le letture dei Dati.

3.1.2 Modellazione del paradigma ECB Mode

Per fornire un confronto con il paradigma CTR, è stato parallelamente modellato il comportamento architetturale del *TEC-Tree* originale. Nel paper originale,[2] gli autori propongono una tecnica di autenticazione denominata *Block Level AREA*, che impiega l'algoritmo AES in modalità *Electronic Code Book* (ECB). In questo schema, il blocco Dati e il suo relativo Contatore (Nonce) sono concatenati e cifrati insieme come una singola entità. Sebbene questo approccio offra il vantaggio teorico di unire confidenzialità e integrità in un unico step, senza l'ausilio di codici MAC esterni, esso impone un gravissimo limite prestazionale. A differenza della generazione anticipata del *Keystream*, possibile con la modalità CTR, la matematica dell'ECB è intrinsecamente "bloccante": l'engine AES non può avviare alcuna permutazione finché l'intero blocco cifrato non sia stato completamente recuperato dalla DRAM. Questa serializzazione forzata rende fisicamente impossibile l'implementazione del *Latency Hiding*.

La nostra modellazione su simulatore riflette esattamente questa barriera sequenziale.

- **Esposizione Totale della Latenza:** Per modellare questo limite fisico, il protocollo di coerenza è stato dotato di un *flag* dinamico (`-is-ecb`) capace di alterare il flusso della macchina a stati. Attivando la modalità ECB, il parallelismo viene annullato e la penalità crittografica completa (*crypto_latency*) viene

inesorabilmente iniettata sul *Critical Path* di **qualsiasi** operazione di memoria, distruggendo l'efficienza sui Dati puri. Nello specifico, l'azione di fetch dei Dati (*sendMemoryRequest*) subisce un ritardo pari alla somma netta del tempo di accesso in RAM e del tempo di elaborazione AES ($T_{DRAM} + T_{AES}$).

L'esecuzione del *workload* in questa modalità permette di quantificare empiricamente l'enorme degrado prestazionale che si avrebbe se si rinunciasse al *Latency Hiding*. Anche per questo scenario, l'estrazione dei risultati automatizzata tramite *batch script* è avviata con il seguente comando:

```
.\4.run\_crypto\_latency\_sweep.sh 0 ECB
```

In questo caso, il secondo parametro (`ECB`) scatena l'iniezione della latenza descritta in precedenza su tutto l'albero e sui blocchi dati del protocollo SLICC.

Al fine di quantificare e osservare visivamente il degrado prestazionale indotto dai due differenti approcci, è stato tracciato il grafico riportato in Figura 3.1.

L'analisi comparativa mostra la dilatazione dei tempi di esecuzione complessivi all'aumentare della latenza crittografica intrinseca dell'hardware, confermando la drastica superiorità del paradigma CTR rispetto alla latenza inesorabilmente esposta dal modello ECB in assenza di meccanismi di esecuzione speculativa.

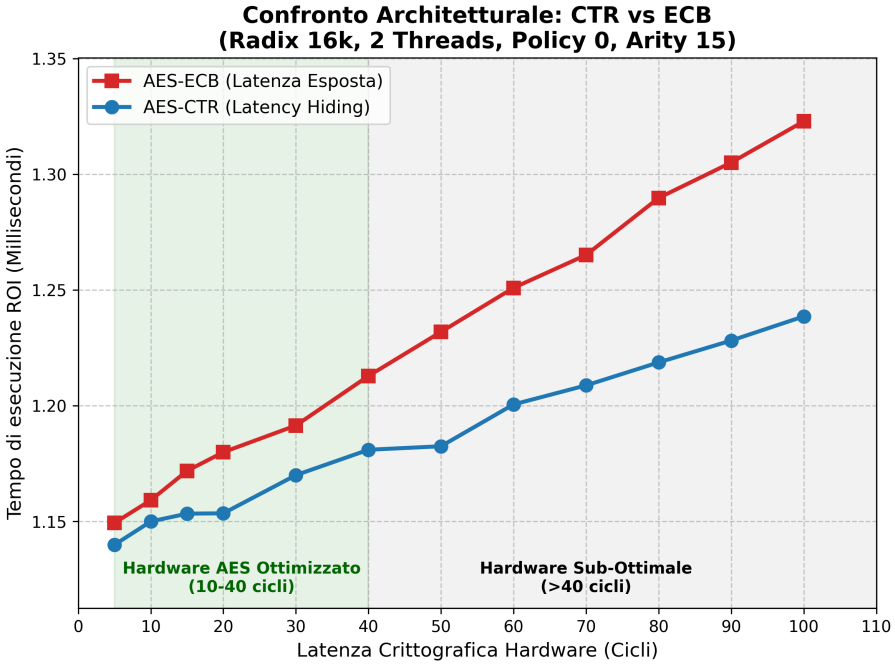


Figura 3.1: Latency Sweep

3.2 Impatto Topologico: Arità e Profondità dell'Albero

Il design di una struttura gerarchica per l'autenticazione della memoria comporta una decisione architetturale critica riguardo alla topologia, di principale interesse per lo studio delle performance. Ciascun albero crittografico infatti è caratterizzato dall'*Arity*, un parametro che permette di definire il numero di nodi figli che dipendono dal singolo nodo genitore. Nel contesto dell'implementazione del TEC-Tree in GEM5, tale parametro rappresenta anche il numero di contatori che possono essere uniti in una singola linea di cache. La variazione di questo parametro definisce un trade-off tra la profondità dell'albero e l'efficienza di utilizzo della cache, con ripercussioni dirette sui tempi di esecuzione. Alberi con arità bassa (Albero Profondo) infatti, riducono il numero di contatori per linea di cache evitando il rischio di overflow degli stessi, ma espongono in caso di *Auth_Miss* a latenze maggiori dovute a percorsi verso la radice a loro volta più profondi. Invece, alberi ad alta arità (Albero Piatto) riducono la profondità dei rami verso la radice. Tale caratteristica si traduce in una riduzione dei costi d'accesso ai Counter Chunk in caso di miss. Tuttavia, forzare Arity troppo alte richiede contatori dimensionalmente più piccoli, esponendo il rischio di overflow e conseguenti onerose operazioni di *re-keying* dell'intera memoria.

3.2.1 Setup Sperimentale

Per validare questo *trade-off* e quantificare il degrado prestazionale causato dalla profondità dell'albero sono stati condotti una serie di esperimenti al variare dell'Arity (testando valori 15, 31, 63). I test sono stati eseguiti sull'algoritmo *Radix-Sort* della suite Splash, variando le dimensioni del dataset per osservare il comportamento sotto diversi livelli di stress della memoria (16384, 65536 e 131072 chiavi) e utilizzando una dimensione di 1MB per l' ultimo livello di cache.
Per poter riprodurre questo esperimento occorre digitare il comando:

```
./5.run_radix_arity_sweep.sh
```

3.2.2 Analisi della Sensitività all'Arity

Osservando i dati riportati nella Tabella 3.1 emerge con chiarezza una correlazione inversa tra l'*Arity* dell'albero e il tempo di esecuzione del workload. Ciò testimonia che la crescita del parametro di Associazione Contatori-Blocco rende effettivamente l'albero meno profondo. Tale teoria è inoltre confermata anche dalle statistiche sulle lettture dei Metadati crittografici (*Counter Chunk*) riportati nella Tabella 3.2, che risultano ridotte all'aumentare dell'Arity poichè minore è la profondità dell'albero e minore sarà il numero di fetch necessari per il recupero dei contatori. Inoltre un fattore di associazione maggiorato si traduce anche in una maggiore efficienza di banda, intesa come porzione di traffico puro per i dati da e verso la memoria RAM, riducendo

il traffico di metadati in condizioni di forte stress a circa il 3% del traffico totale (vedi Tabella 3.3).

Array Size	**Arity 15**	**Arity 31**	**Arity 63**
16 384	2 356 104 000	2 349 061 000	2 345 624 500
65 536	9 001 416 500	8 993 170 000	8 989 932 500
131 072	17 860 252 000	17 853 201 000	17 848 827 500

Tabella 3.1: Sensibilità all'Arità — *ROI Ticks*

Array Size	**Arity 15**	**Arity 31**	**Arity 63**
16 384	476	282	181
65 536	914	487	280
131 072	1542	768	427

Tabella 3.2: Sensibilità all'Arità — *Letture di Counter Chunk*

Array Size	**Arity 15**	**Arity 31**	**Arity 63**
16 384	89,01%	92,84%	95,16%
65 536	91,95%	95,36%	97,22%
131 072	93,21%	96,26%	97,83%

Tabella 3.3: Sensibilità all'Arità — *Bandwidth Efficiency*

Sebbene l'analisi di sensitività abbia palesato gli evidenti vantaggi prestazionali offerti da un'arietà elevata (come 31 o 63) dovuti alla minore profondità dell'albero e all'abbattimento dell'overhead sui metadati, in fase di progettazione si è deciso di mantenere un'impostazione più conservativa, fissando l'arità a 15 per tutti gli esperimenti e le analisi successive. Questa scelta architetturale rappresenta un compromesso fondamentale tra le prestazioni pure e i vincoli strutturali dettati dal protocollo crittografico, infatti aumentare eccessivamente il fatto-

re di ramificazione impone di frammentare il singolo blocco cache dei metadati (tipicamente da 64 byte) in un numero maggiore di contatori figli. Di conseguenza, un'arità più elevata riduce drasticamente il numero di bit disponibili per la memorizzazione di ogni singolo contatore. Contatori di dimensioni ridotte raggiungono la loro massima capacità (*overflow*) molto più rapidamente. Quando si verifica un *counter overflow* a livello hardware, per evitare attacchi crittografici (come il *replay attack*), l'architettura è costretta ad aggiornare la chiave o, nei casi peggiori, ad innescare una procedura di re-cifratura (re-encryption) totale dell'intera regione di memoria protetta. Si tratta di un'operazione dal costo computazionale e temporale catastrofico, che andrebbe a vanificare istantaneamente qualsiasi guadagno prestazionale accumulato grazie all'appiattimento dell'albero.

Pertanto fissare l'*Arity* a 15 si è rivelata la scelta ideale per architettura proposta, garantendo una profondità tale da non congestionare severamente la larghezza di banda e al contempo permette ai contatori di ridurre la probabilità di overflow durante il normale ciclo di esecuzione del sistema.

3.3 Strategie di Evizione ed Impatto Prestazionale

La statistica più interessante che emerge dagli esperimenti condotti nella sezione precedente riguarda il numero di scritture verso la DRAM prodotte all'aumentare delle dimensioni del workset (131K). Come si evince dai risultati, l'aumento dell'arity riduce a sua volta il traffico di metadati verso la DRAM e di conseguenza il numero di stalli per l'autenticazione. Tuttavia è fondamentale comprendere le dinamiche interne alla cache che generano tale traffico. In un'architettura di sicurezza come il TEC-Tree, Dati e Metadati (Counter Chunks) condividono e competono per lo stesso spazio fisico all'interno della Last Level Cache (LLC). Quando il Working Set supera la capienza della cache (come negli esperimenti ad Array size di 131K), le fisiologiche operazioni di rimpiazzo rischiano di innescare fatali *Capacity Miss* sui contatori padri. Infatti, se un blocco (Dato o Metadato) "sporco" viene sfrattato verso la RAM e il suo rispettivo contatore genitore non è più presente in LLC, si verifica un *Auth Miss*: la transizione di *writeback* viene immediatamente bloccata, imponendo un oneroso stallo alla pipeline per il recupero del nodo padre dalla memoria principale.

Arity	Evizioni Pulite	Evizioni Sporche	Auth Miss (Stalli)	Totale
15	951	1853	39	2873
31	856	1234	8	2098
63	746	1060	13	1819

Tabella 3.4: Evizioni LLC (Dati vs Metadati) per array di dimensioni pari a 131K

Come si evince dalla Tabella 3.4, sebbene il volume complessivo delle evizioni (sia pulite che sporche) sia nell'ordine delle migliaia, il numero di stalli (*Auth Miss*) registrato dal simulatore rimane sorprendentemente basso, sfiorando appena il 2% del totale delle evizioni sporche. Questo risultato estremamente positivo non è casuale, ma è il frutto diretto delle strategie di ritenzione *Topology-Aware* integrate nel *Memory Controller*. L'obiettivo delle sezioni seguenti è quello di descrivere le tre diverse policy di rimpiazzo introdotte in questo progetto (Baseline, Strict e Tectree-Optimized), analizzando come l'alterazione delle tradizionali regole di evizione (*Least Recently Used*) permetta di proteggere statisticamente i nodi più alti dell'albero crittografico, garantendo un deflusso rapido per le operazioni di *writeback*.

3.3.1 Policy 0: Strict Policy

La Policy 0, o *Strict Policy*, rappresenta un approccio strategico di protezione estremo per i metadati. L'obbiettivo è quello di evitare gli *Auth_ Miss*, conferendo ai metadati una priorità di ritenzione assoluta. Ciò si ottiene promuovendo a blocco *Most Recently Used* il metadato ogni volta che vi è una hit sul Data chunk ad esso associato e quando

questo blocco (Counter chunk) viene allocato all'interno della cache di ultimo livello. Sebbene questa strategia, ottenuta attraverso il *pinning* dei metadati, riesca a trattenere l'albero crittografico on-chip, genera però un effetto collaterale non trascurabile, noto come "*Data Starvation*". I blocchi Dati, impossibilitati a scavalcare i metadati nella gerarchia MRU, subiscono evizioni immediate (thrashing continuo), degradando drammaticamente l'hit rate dei dati applicativi.

3.3.2 Policy 1: Baseline

La Policy 1, o *Baseline*, rappresenta invece un approccio conservativo in cui i metadati "invecchiano" naturalmente all'interno della cache e che offre invece ai dati la possibilità, a seguito di hit, di essere promossi come blocco *Most Recently Used*. In questo scenario, quando il working set dell'applicazione supera la capienza dell'ultimo livello di cache, l'intenso traffico di dati spinge rapidamente i Counter chunks verso le posizioni di *Least Recently Used block*, condannandoli ad evizioni precoci. Tale politica, pur essendo molto semplice da implementare a livello hardware, si rileva incompatibile con il TEC Tree: l'assenza di una policy *Tree-Aware* infatti, aumenta la probabilità di *Auth_Miss* e i conseguenti stalli sull'intera catena di fiducia.

3.3.3 Policy 2: TEC-Tree-Optimized

La Policy 2, o *TEC-Tree-Optimized*, costituisce il vero contributo architetturale di questo progetto e propone un approccio *topology-aware* per garantire la ritenzione dei metadati senza penalizzare estremamente i dati applicativi. Sfruttando le proprietà strutturali e la matematica del TEC-Tree, il Memory Controller è in grado di riconoscere il livello gerarchico $L_i \in \{L_1, L_2, \ldots, L_{\text{Root}}\}$ a cui appartiene un determinato *Counter chunk*. Al momento dell'accesso in cache, l'algoritmo SLICC non aggiorna incondizionatamente lo stato *Least Recently Used* (LRU), ma campiona una variabile aleatoria per applicare una promozione probabilistica alla posizione *Most Recently Used* (MRU).

Formalmente, la probabilità $P(\text{MRU})$ di proteggere un metadato di livello i è definita dalla seguente legge empirica implementata a livello hardware[1]:

[1]Più alto è il livello gerarchico del *Counter chunk*, maggiore sarà il numero di blocchi ad esso associati.

$$P(\text{MRU} \mid L_i) = \begin{cases} 30\% & \text{se } i = 1 \text{ (Livello Base)} \\ 40\% & \text{se } i = 2 \\ 50\% & \text{se } i = 3 \\ 60\% & \text{se } i = 4 \\ 80\% & \text{se } i = 5 \\ 100\% & \text{se } i = 6 \text{ (Prossimità alla Radice)} \end{cases} \tag{3.1}$$

Da un punto di vista implementativo, tale distribuzione è ottenuta calcolando dinamicamente la condizione booleana:

$$\text{rand}() \bmod 10 < \tau_i \tag{3.2}$$

dove il valore di soglia τ_i cresce monotonicamente risalendo i rami dell'albero verso la radice globale. Tale formulazione garantisce che i nodi vicini alle foglie, nonché i livelli più bassi dell'albero, abbiano una probabilità minore di promozione in cache e una minore probabilità di inquinare i set della cache, garantendo più "respiro" ai dati e riducendo il fenomeno della *starvation*. Al contrario, i nodi prossimi alla radice, ovvero i livelli più alti, godono di una protezione statistica estremamente aggressiva, scongiurando l'occorrenza di *Auth_Miss* e stalli gerarchicamente più onerosi.

3.3.4 Analisi Comparativa dei risultati

Al fine di motivare e valutare l'impatto architetturale delle scelte prese in considerazione, è stato condotta un'estesa *Design Space Exploration* (DSE) incrociando tre fattori chiave: il fattore di ramificazione dell'albero (*Arity* 15, 31, 63), le dimensioni del workload (*Array Size*), e le tre politiche di rimpiazzo MRU proposte (Policy 0: *Strict*, Policy 1: *Baseline*, Policy 2: *TEC-Tree-Optimized*).

Per garantire che la gerarachia di memoria operasse in condizione di forte stress e massima saturazione, scatenando una grande mole di *eviction*, l' analisi comparativa mostrata in figura 3.2 si concentra sugli esperimenti eseguiti su Radix Sort con array size fissata 131K chiavi. Questa dimensione infatti ci permette di osservare molto chiaramente l'intensa competizione che si genera nell' ultimo livello di cache tra i dati applicativi ed i metadati. Osservando il primo grafico ("Impatto prestazionale delle Policy MRU"), emerge un trend inequivocabile: all'aumentare dell'Arity, l'albero si appiattisce riducendo le latenze di attraversamento a prescindere dalla policy scelta. La metrica più interessante tuttavia, si ottiene incrociando in configurazione un'Arity di 63 e la Policy 2. Tale setup registra infatti il tempo di esecuzione minore in assoluto (17.731.164.000 Ticks), superando sia la gestione ingenua (Baseline) che la policy 0 (*Strict*). La motivazione di questo vantaggio prestazionale diventa evidente analizzando i due grafici sottostanti, che illustrano il numero di evizioni dalla LLC. La **Poli-**

cy 0 (Strict), blindando incondizionatamente i metadati in cache (0 evizioni di metadati, come visibile nel secondo grafico), monopolizza lo spazio disponibile, portando al valore massimo di sfratti sui Dati Applicativi (2834, 2090 e 1806 sfratti, nel terzo grafico) causando l'effetto indesiderato noto come *starvation* dei dati. Al contrario, la **Policy 1 (Baseline)** tratta allo stesso modo dati e metadati: questo favorisce i dati Applicativi (minor numero di evizioni), ma causa un tracollo per i metadati, che vengono sfrattati fino a 125 volte (Arity 15). Questo fenomeno è estremamente penalizzante, poiché recuperare un nodo metadato vicino alla radice genera una latenza a cascata per l'intera verifica di integrità. In questo scenario, la **Policy 2 (TEC-Tree-Optimized)** dimostra di trovare il *trade-off* ottimale:

- **Riduzione drastica dei Miss critici:** Rispetto alla Baseline, la Policy 2 abbassa le evizioni dei metadati di oltre il 50% (passando da 125 a 54 per Arity 15 e da 35 a 11 per Arity 63), schermando con successo i nodi superiori del TEC-Tree.

- **Impatto marginale sui Dati Applicativi:** A fronte di questa massiccia protezione dei metadati, gli sfratti dei Dati Applicativi subiscono solo un lieve e del tutto trascurabile incremento rispetto alla Baseline, mantenendosi ben lontani dai picchi disastrosi della Policy 0.

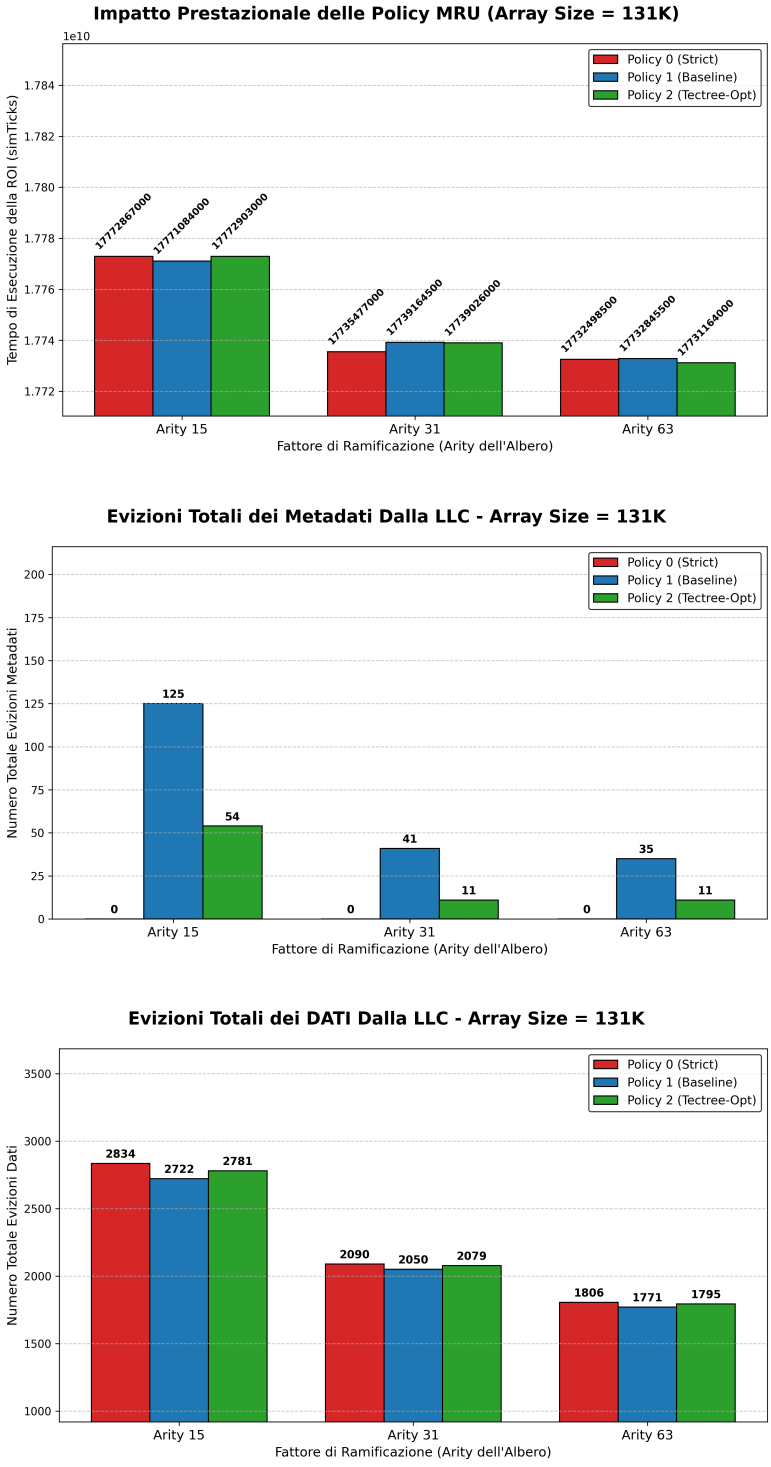


Figura 3.2: Confronto delle prestazioni al variare dell'arità dell'albero

3.4 TEC-Tree a confronto con TARDIS-TSO e MESI

Per poter validare l'efficacia delle ottimizzazioni proposte e quantificare il reale contributo architetturale del progetto, questa sezione presenta un'analisi comparativa delle prestazioni. Il protocollo TARDISTSO TEC-Tree viene posto a diretto confronto con un' architettura di riferimento operante su una gerarchia di memoria a due livelli (L1/L2), ovvero il protocollo di coerenza MESI, oltre che con il preimplementato protocollo Tardis-TSO. L'obbiettivo principale di questo benchmarking è dimostrare come l'introduzione di un ulteriore livello di cache per dati e metadati (mixed cache) unita ad un'opportuno meccanismo di protezione dei metadati (Policy 2) permetta non solo di soddisfare i requisti di sicurezza della memoria ma al contempo di ridurre l'overhead legato alla sua autenticazione. Come si evince dall'andamento prestazionale riportato in figura 3.5, che mostra i tempi d'esecuzione e il numero di richieste verso la memoria al variare delle dimensioni dell'array, l'integrazione del TEC-Tree introduce un'inevitabile "tassa" architetturale. Tale overhead risulta tuttavia fisiologico nelle architetture *Secure Memory*. Il fattore preponderante di quest'analisi prestazionale è sicuramente il limite fisico dell'hardware preposto per la cifratura. Nel nostro modello simulato (TARDITSO-TEC-Tree) ogni operazione di lettura o scrittura verso la memoria sconta una la-

tenza intrinseca (10 cicli di clock). Ogni *Cache Miss* che coinvolge un blocco dati costringe la Directory ad uno stato di stallo per quella linea di cache. Ed è proprio in questo contesto che il connubio tra una *Policy* Topology-aware e la scelta di un *Arity* bilanciata diventa di fondamentale importanza per mascherare la latenza e mantenere l'overhead a livelli accettabili.

Inoltre un'ulteriore considerazione merita la scelta della modalità crittografica implementata. Il modello attuale simula una codifica ECB (*Electronic Codebook*). Sebbene in letteratura l'ECB sia talvolta considerato un meccanismo di cifratura debole esso presenta un vantaggio architetturale formidabile: l'assenza di dipendenze logiche tra le sezioni dello stesso blocco. Con tale premessa infatti è possibile ridurre drasticamente la latenza crittografica implementando batterie di motori AES paralleli in hardware, capaci di cifrate e decifrare blocchi. Invece, tecniche avanzate come il passaggio a modalità *Counter Mode* (CTR) che permette di pre-calcolare i pad crittografici mascherando del tutto la latenza di cifratura rappresentano le direzioni future più promettenti per azzerare quasi totalmente questo limite architetturale.

Array	Protocollo	Ticks (ROI)	Richieste DRAM	Richieste LLC/Dir	Overhead
16.384	MESI (2 Livelli)	1.516.319.000	3.378	3.831	*Baseline*
	TARDIS-TSO	1.773.384.000	22.005	20.975	+16.95%
	TARDIS-TSO TEC-Tree	2.356.112.000	3.853	432.489	+55.38%
65.536	MESI (2 Livelli)	5.702.614.000	9.522	9.975	*Baseline*
	TARDIS-TSO	6.698.350.500	83.930	77.573	+17.46%
	TARDIS-TSO TEC-Tree	9.001.348.000	10.435	1.716.764	+57.85%
131.072	MESI (2 Livelli)	11.286.742.500	18.479	18.201	*Baseline*
	TARDIS-TSO	13.289.184.500	166.360	153.090	+17.74%
	TARDIS-TSO TEC-Tree	17.860.314.000	21.186	3.428.947	+58.24%

Tabella 3.5: Riepilogo delle prestazioni nella ROI per il Benchmark Radix.

Capitolo 4

Conclusioni e Sviluppi futuri

4.1 Conclusioni

L'obbiettivo principale di questo lavoro di tesi è stato quello di analizzare, implementare e valutare nuove strategie architetturali per mitigare il collo di bottiglia introdotto dai meccanismi di autenticazione della memoria (Secure Memory) . L'overhead legato al recupero e alla verifica dei metadati di sicurezza (come i nodi dell' albero di integrità) rappresenta infatti uno dei limiti più critici per la scalabilità dei moderni sistemi protetti. In questo contesto, il lavoro si è concentrato sull'implementazione e l'ottimizzazione del *TEC-Tree* all'interno del simulatore architetturale *gem5*. Il contributo si è articolato su due direttrici principali:

- **Supporto all'Arità Dinamica:** È stata rimodellata la logica del Directory Controller e i file di configurazione Python-SLICC per supportare alberi di integrità con arità configurabile a *run-time*. I test effettuati al variare dell'arità (es. 15, 31, 63) hanno dimostrato come l'appiattimento dell'albero influisca positivamente sui tempi di esecuzione, riducendo i livelli di ricorsione necessari per la verifica.

- **Policy di Cache Topology-Aware:** È stata introdotta una politica di rimpiazzo custom (Policy 2) per l'ultimo Livello di Cache (LLC). Piuttosto che affidarsi a un classico algoritmo LRU incondizionato, la policy implementata assegna dinamicamente e in modo probabilistico lo stato di *Most Recently Used* (MRU) ai *Counter chunk*, basandosi sulla loro posizione gerarchica all'interno dell'albero. Questo ha permesso di schermare e proteggere i nodi più vicini alla radice dal fenomeno dell'evizione, riducendo drasticamente il costo dei *cache miss* più penalizzanti (Auth_Miss).

I risultati sperimentali, ottenuti tramite una suite automatizzata di *design space exploration* sul benchmark parallelo Radix, hanno confermato la validità dell'approccio. Le simulazioni hanno dimostrato che un bilanciamento accurato tra l'arità dell'albero e una policy di caching *topology-aware* garantisce una sensibile riduzione dei tempi

d'esecuzione nella Regione di Interesse (ROI), confermando un utilizzo più efficiente della gerarchia di memoria.

4.2 Sviluppi Futuri

Nonostante i promettenti risultati ottenuti, il paradigma esplorato in questo lavoro si presta a diverse estensioni future:

- **Validazione su workload eterogenei:** Le attuali valutazioni sono state condotte su micro-benchmark mirati (Radix Sort). Uno dei progressi naturali consiste nell'estendere le simulazioni a suite di benchmark standard e complessi (come *PARSEC*), per valutare il comportamento delle policy di sicurezza in presenza di pattern di accesso alla memoria irregolari e altamente concorrenti;

- **Stima dei costi Hardware (RTL):** Poichè l'algoritmo probabilistico per la promozione in MRU richiede la generazione di numeri pseudo-casuali e logica comparativa nel Memory Controller, un'analisi in linguaggio di descrizione hardware (es. Verilog/VHDL) permetterebbe di quantificare l'esatto overhead in termini di area e dissipazione di potenza introdotto dalla *Policy 2*;

- **Testing del Protocollo in FS**: risolvere le criticità emerse durante il lavoro per promuovere un'esecuzione in ambiente di

sviluppo in modalità full system e verificare il comportamento del nuovo protocollo in funzione di un carico reale come l'esecuzione di un sistema operativo;

- **Arità Eterogenea:** Il framework attualmente supporta un'arità globale. Una futura estensione potrebbe prevedere alberi asimmetrici, in cui i livelli vicini alla radice hanno un'arità differente da quelli vicini alle foglie, al fine di massimizzare il riempimento delle linee di cache e minimizzare il *padding* o la frammentazione interna.

Bibliografia

[1] Emanuele Di Maio. Sviluppo di un protocollo di coerenza di cache expiration-based nel simulatore gem5. Master's thesis, Università degli Studi di Napoli Federico II, Napoli, Italia, 2025. Corso di Laurea Magistrale in Ingegneria Informatica, Relatore: Prof. Alessandro Cilardo.

[2] Reouven Elbaz, David Champagne, Ruby B. Lee, Lionel Torres, Gilles Sassatelli, and Pierre Guillemin. Tec-tree: A low-cost, parallelizable tree for efficient defense against memory replay attacks. In *Cryptographic Hardware and Embedded Systems – CHES 2007*, volume 4727 of *Lecture Notes in Computer Science*, pages 289–302. Springer-Verlag Berlin Heidelberg, 2007.

[3] Blaise Gassend, G. Edward Suh, Dwaine Clarke, Marten van Dijk, and Srinivas Devadas. Caches and hash trees for efficient memory integrity verification. In *Proceedings of the Ninth International Symposium on High-Performance Computer Architecture (HPCA-9'03)*, pages 153–164. IEEE Computer Society, 2003.

[4] Shay Gueron. A memory encryption engine suitable for general purpose processors. Cryptology ePrint Archive, Paper 2016/204, 2016.

[5] Mario Werner, Thomas Unterluggauer, Robert Schilling, David Schaffenrath, and Stefan Mangard. Transparent memory encryption and authentication. In *Proceedings of the Conference on Design, Automation and Test in Europe (DATE)*. Graz University of Technology, 2016.

[6] Xiangyao Yu, Hongzhe Liu, Ethan Zou, and Srinivas Devadas. Tardis 2.0: Optimized time traveling coherence for relaxed consistency models. In *2016 International Conference on Parallel Architecture and Compilation Techniques (PACT)*, pages 241–254. IEEE, 2016.